our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Tulpea

SECURITY REVIEW

Date: 28 March 2026

CONTENTS

1. About Shieldify Security **3**

2. Disclaimer **3**

3. About Tulpea **3**

4. Risk classification **3**

4.1 Impact 3

4.2 Likelihood 4

5. Security Review Summary **4**

5.1 Protocol Summary 4

5.2 Scope 4

6. Findings Summary **4**

7. Findings **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, for which this report is intended, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Tulpea

ERC-4626 tokenized vault with ERC-7540 asynchronous redemptions and a pluggable strategy registry. Solidity 0.8.24, OpenZeppelin Contracts v5.4.0, UUPS proxy pattern. Asset: configurable at initialization (currently USDT0, 6 decimals on MegaETH).

Trusted Roles & Admin Powers

Actor	Description
User	Signs an intent expressing swap parameters and minimum output
Executor	Whitelisted address that submits intents + swap calldata
Treasury	Receives surplus above user's minimum output

Learn more about Tulpea's concept and the technicalities behind it: README.md

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users

- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 5 days with a total of 80 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and three Low severity issues. They're related to access control gaps, missing validation checks and unsafe parameter handling.

The Tulpea team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	**Tulpea**
Repository	Tulpea/Vault
Type of Project	ERC-4626, ERC-7540
Security Review Timeline	5 days
Review Commit Hash	64d394908900fe692e5a2036f1d8c00af6b326db
Fixes Review Commit Hash	d4b867ecca118bf60270bdcc00452f412697d776

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
TulpeaYieldVault.sol	499
strategies/RealEstateStrategy.sol	261
strategies/AvonStrategy.sol	168
Total	928

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **1**
- **Low** issues: **3**
- **Info** issues: **6**

ID	Title	Severity	Status
[M-01]	The `processReport()` Health Check Bypassed on First Report	Medium	Fixed
[L-01]	Operator Can Reassign Redeem Controller and Lock Out Original Owner	Low	Fixed
[L-02]	Zero `minOut` for Dust USDm Amounts Allows 100% Slippage Loss in `_swapUsdmToUsdt()`	Low	Fixed
[L-03]	Emergency NFT Recovery Allows Extraction of Active Portfolio Positions	Low	Fixed
[I-01]	Missing Explicit `approve(0)` Sanitization Before `forceApprove()`	Info	Fixed
[I-02]	Misleading Custom Error Name in `resolveEmergencyShutdown()`	Info	Fixed
[I-03]	`FundsDeployedToStrategy` Event Defined but Never Emitted	Info	Fixed
[I-04]	`requestDeploy()` and `executeDeploy()` Are Not Blocked During Emergency Shutdown	Info	Fixed
[I-05]	Inconsistency in How the Pausing Is Handled in Redeeming Processes	Info	Fixed
[I-06]	Lack of Zero Address Check in `withdraw()`, `redeem()` and `_claimWithdrawal()` Functions	Info	Fixed

7. Findings

[M-01] The `processReport()` Health Check Bypassed on First Report

Severity

Medium Risk

Description

The `processReport()` enforces `maxProfitReportBps` deviation bounds only when `previousDebt > 0`. Because `addStrategy` initializes `currentDebt = 0`, the first `processReport()` call for every strategy skips health checks and accepts any reported `totalAssets()` value without a bound.

As a result, a strategy that holds assets before vault deployment (for example, airdropped tokens or residual yield) can increase `totalDebt` and vault NAV by the full reported amount on the first report, despite health checks being enabled.

Attack flow: 1. Owner adds strategy `S` and enables a tight health check. 2. `S` has non-zero `totalAssets()` before any debt is deployed. 3. Keeper calls `processReport(S)` with

`previousDebt = 0`. 4. `if (healthCheckEnabled && previousDebt > 0)` is false, so bounds are skipped. 5. `currentAssets - 0` is fully recognised as profit and added to `totalDebt`.

Location of Affected Code

File: TulpeaYieldVault.sol#L544

```
function addStrategy(address strategy) external onlyOwner {
    // code
    strategies[strategy] = StrategyConfig({isActive: true, currentDebt:
        0, lastTotalAssets: 0});
    strategyList.push(strategy);
    emit StrategyAdded(strategy);
}
```

File: TulpeaYieldVault.sol#L596

```
function processReport(address strategy) external onlyKeeperOrOwner
    nonReentrant returns (uint256 profit, uint256 loss) {
    // code
    uint256 previousDebt = config.currentDebt;

    // Health check: validate deviation bounds before applying profit/
        loss
    if (healthCheckEnabled && previousDebt > 0) {
        if (currentAssets > previousDebt) {
            uint256 profitDelta = currentAssets - previousDebt;
            uint256 maxAllowed = (previousDebt * maxProfitReportBps) /
                MAX_BPS;
            if (profitDelta > maxAllowed) revert HealthCheckFailed(
                profitDelta, maxAllowed, false);
        }
        // code
    }

    if (currentAssets > previousDebt) {
        profit = currentAssets - previousDebt;
        config.currentDebt = currentAssets;
        config.lastTotalAssets = currentAssets;
        totalDebt += profit;
    }
    // code
}
```

Impact

The `maxProfitReportBps` health check provides no protection at the zero-debt baseline. On a 100,000 USDT vault with a 1% cap configured, a 2,000 USDT airdrop to a zero-debt strategy inflates the NAV by 2% in a single `processReport()` call. The policy the operator configured is silently void. The bypass repeats on every strategy's first report and after any full debt drain cycle.

Proof of Concept

```
it("demonstrates health check is skipped on first processReport()
   allowing unbounded NAV inflation", async function () {
    // 1) Alice deposits 100,000 USDT — establishes real TVL baseline.
    await vault.connect(alice).deposit(ALICE_DEPOSIT, aliceAddr);
    const aliceShares = await vault.balanceOf(aliceAddr);
    expect(aliceShares).to.equal(ALICE_DEPOSIT * SHARE_SCALE, "Alice must
        hold correct share amount");

    // 2) Configure health check: 1% max profit, health check enabled.
    //    At this cap a 1,000 USDT jump on a 100k base would be the
       maximum allowed,
    //    and 2,000 USDT should revert with HealthCheckFailed.
    await vault.connect(admin).setHealthCheck(100, 100, true);
    expect(await vault.healthCheckEnabled()).to.equal(true);
    expect(await vault.maxProfitReportBps()).to.equal(MAX_PROFIT_BPS);

    // 3) Add the strategy. addStrategy always initialises currentDebt =
       0.
    await vault.connect(admin).addStrategy(await mockStrategy.getAddress
       ());
    const config = await vault.strategies(await mockStrategy.getAddress()
       );
    expect(config.currentDebt).to.equal(0n, "currentDebt must be 0 — no
       funds deployed yet");

    // 4) Strategy receives 2,000 USDT via airdrop before any vault funds
        are deployed.
    //    previousDebt = 0, so profitDelta = 2,000 USDT.
    //    With 1% maxProfitReportBps: maxAllowed = (0 * 100) / 10000 = 0.
    //    Any non-zero profit should trigger HealthCheckFailed — but the
       guard
    //    short-circuits on previousDebt > 0, so it never fires.
    await mockStrategy.setBalance(STRATEGY_DONATION);

    // Snapshot: vault NAV before the bypassed report
    const navBefore = await vault.totalAssets();
    expect(navBefore).to.equal(ALICE_DEPOSIT, "NAV before report must
       equal Alice's deposit");
    expect(await vault.totalDebt()).to.equal(0n);
```

```
    // 5) Keeper calls processReport().
    //    EXPECTED (if health check worked): revert HealthCheckFailed
       (2000e6, 0, false)
    //    ACTUAL: no revert — 2,000 USDT accepted unconditionally.
    await expect(
      vault.connect(keeper).processReport(await mockStrategy.getAddress()
         )
    ).to.not.be.reverted;

    // 6) Vault NAV jumped by the full 2,000 USDT donation — 2% inflation
        in one tx.
    //    The configured 1% cap provided zero protection.
    const navAfter = await vault.totalAssets();
    expect(navAfter).to.equal(
      ALICE_DEPOSIT + STRATEGY_DONATION,
      "NAV must reflect full 2,000 USDT injection — health check bypassed
         "
    );

    // 7) totalDebt absorbed the full donation with no bound check.
    expect(await vault.totalDebt()).to.equal(
      STRATEGY_DONATION,
      "totalDebt must equal donation — 2,000 USDT accepted, 1% cap
         ignored"
    );

    // 8) Alice's 100,000 USDT deposit is now redeemable for ~102,000
       USDT.
    //    A future depositor entering at the new NAV pays the inflated
       price,
    //    subsidising Alice's windfall. The health check policy is void.
    const aliceRedeemValue = await vault.convertToAssets(aliceShares);
    expect(aliceRedeemValue).to.be.greaterThan(
      ALICE_DEPOSIT,
      "Alice's position must be worth more than her deposit after the
         unguarded NAV jump"
    );
    const aliceWindfall = aliceRedeemValue - ALICE_DEPOSIT;
    expect(aliceWindfall).to.be.greaterThanOrEqual(
      STRATEGY_DONATION - 1n,
      "Alice's windfall must equal the full 2,000 USDT donation (±1 wei
         rounding) — no health check fired"
    );
  });
```

Recommendation

Remove the `previousDebt > 0` gate. When `previousDebt == 0`, use `totalAssets()` (or a constant floor of 1) as the reference base to preserve meaningful deviation bounds:

```
if (healthCheckEnabled) {
    if (currentAssets > previousDebt) {
        uint256 profitDelta = currentAssets - previousDebt;
        uint256 base = previousDebt > 0 ? previousDebt : totalAssets() +
            1;
        uint256 maxAllowed = (base * maxProfitReportBps) / MAX_BPS;
        if (profitDelta > maxAllowed) revert HealthCheckFailed(
            profitDelta, maxAllowed, false);
    } else if (currentAssets < previousDebt) {
        uint256 lossDelta = previousDebt - currentAssets;
        uint256 maxAllowed = (previousDebt * maxLossReportBps) / MAX_BPS;
        if (lossDelta > maxAllowed) revert HealthCheckFailed(lossDelta,
            maxAllowed, true);
    }
}
```

This ensures the health check fires on the first report and on every report after a full debt drain, matching the operator's configured intent.

Team Response

Fixed.

[L-01] Operator Can Reassign Redeem Controller and Lock Out Original Owner

Severity

Low Risk

Description

A delegated operator can call `requestRedeem()` for an owner while supplying an arbitrary `controller`. Pending redemption state is keyed by `controller`, not by `owner`, and `cancelWithdraw()` only reads `pendingWithdrawalShares[msg.sender]`. This allows an authorized operator to escrow a victim's shares into a hostile controller slot that the victim cannot cancel.

If the admin later calls `fulfillRedeem(hostileController)`, assets become claimable under that hostile controller key and can be withdrawn by that controller (or its operator), while the original owner cannot claim from that flow.

Location of Affected Code

File: TulpeaYieldVault.sol#L389

```
function requestRedeem(
    uint256 shares,
    address controller,
    address owner
) external nonReentrant whenNotPaused returns (uint256 requestId) {
    if (shares == 0) revert ZeroShares();
    if (controller == address(0)) revert ZeroAddress();
    if (pendingWithdrawalShares[controller] > 0) revert
        WithdrawalAlreadyPending();

    // Auth: msg.sender must be owner or operator of owner
    if (msg.sender != owner && !isOperator(owner, msg.sender)) revert
        Unauthorized();

    _transfer(owner, address(this), shares);

    pendingWithdrawalShares[controller] = shares;
    pendingWithdrawalOwner[controller] = owner;
    totalEscrowedShares += shares;
}
```

File: TulpeaYieldVault.sol#L418

```
function cancelWithdraw() external nonReentrant whenNotPaused {
    uint256 shares = pendingWithdrawalShares[msg.sender];
    if (shares == 0) revert NothingPending();
    // code
}
```

File: TulpeaYieldVault.sol#L442

```
function fulfillRedeem(address user) external onlyOwner nonReentrant {
    uint256 shares = pendingWithdrawalShares[user];
    // code
    claimableWithdrawals[user] += assets;
    claimableWithdrawalShares[user] += shares;
}
```

File: TulpeaYieldVault.sol#L860

```
function _claimWithdrawal(address owner, address receiver, uint256 assets
    , uint256 shares) internal {
    if (msg.sender != owner && !isOperator(owner, msg.sender)) revert
        Unauthorized();
    // code
    IERC20(asset()).safeTransfer(receiver, assets);
}
```

File: TulpeaYieldVault.sol#L89

File: TulpeaYieldVault.sol#L110

Impact

Loss of control over an escrowed redemption, and potential permanent loss for the original owner if `fulfillRedeem()` is executed for the attacker-selected controller.

Proof of Concept

```
it("demonstrates operator-controlled hostile controller can steal
    redeemed assets", async function () {
  // 1) Victim deposits assets and receives shares.
  await vault.connect(victim).deposit(ASSETS, victimAddr);
  const victimShares = await vault.balanceOf(victimAddr);
  expect(victimShares).to.equal(ASSETS * SHARE_SCALE);

  // 2) Victim sets attacker as operator.
  await vault.connect(victim).setOperator(attackerAddr, true);
  expect(await vault.isOperator(victimAddr, attackerAddr)).to.equal(true)
      ;

  // 3) Attacker requests redeem with controller=hostileController, owner
      =victim.
  await vault.connect(attacker).requestRedeem(victimShares,
      hostileControllerAddr, victimAddr);
  expect(await vault.pendingWithdrawalShares(hostileControllerAddr)).to.
      equal(victimShares);
  expect(await vault.balanceOf(victimAddr)).to.equal(0);

  // 4) Victim cancelWithdraw() reverts NothingPending (pending is under
      hostile controller).
  await expect(vault.connect(victim).cancelWithdraw())
    .to.be.revertedWithCustomError(vault, "NothingPending");

  // 5) Admin fulfills against hostile controller slot.
  await vault.connect(admin).fulfillRedeem(hostileControllerAddr);
  const hostileClaimable = await vault.claimableWithdrawals(
      hostileControllerAddr);
  expect(hostileClaimable).to.equal(ASSETS);
```

```
  // 6) Hostile controller withdraws claimable assets to itself.
  const hostileBalanceBefore = await usdc.balanceOf(hostileControllerAddr
      );
  await vault.connect(hostileController).withdraw(hostileClaimable,
      hostileControllerAddr, hostileControllerAddr);
  const hostileBalanceAfter = await usdc.balanceOf(hostileControllerAddr)
      ;
  expect(hostileBalanceAfter - hostileBalanceBefore).to.equal(ASSETS);

  // 7) Victim cannot claim assets from this redemption flow.
  await expect(vault.connect(victim).withdraw(ASSETS, victimAddr,
      victimAddr))
    .to.be.revertedWithCustomError(vault, "NothingClaimable");
  expect(await vault.claimableWithdrawals(victimAddr)).to.equal(0);
});
```

Recommendation

Constrain the controller target in `requestRedeem()` so an operator cannot redirect another user's escrow into an unrelated controller slot.

```
if (controller != owner && controller != msg.sender) revert
    InvalidController();
```

Team Response

Fixed.

[L-02] Zero `minOut` for Dust USDm Amounts Allows 100% Slippage Loss in `_swapUsdmToUsdt()`

Severity

Low Risk

Description

The `AvonStrategy._swapUsdmToUsdt()` computes `amountOutMinimum` by applying slippage and then downscaling from 18 decimals (USDm) to 6 decimals (asset).

For sufficiently small `amountIn`, integer flooring makes `minOut == 0`, so the swap executes with no output floor. This allows a worst-case execution of `0` output units for dust-sized swaps, including adversarial execution around the transaction.

The path is reachable during normal withdrawals: 1. `withdraw()` redeems USDm from MegaVault. 2. Any non-zero redeemed amount is forwarded to `_swapUsdmToUsdt()`. 3. If the redeemed amount is below the dust threshold, `amountOutMinimum` becomes `0`.

Threshold (current constants): - `DECIMAL_SCALE = 1e12` - `MAX_SLIPPAGE_BPS = 50` - `minOut = floor(amountIn * 9950 / 10000 / 1e12)` - `minOut == 0` for `amountIn < 1,005,025,125,629` (USDm wei), i.e. approximately `< 0.000001005025125629 USDm`
.

Location of Affected Code

File: AvonStrategy.sol#L323-L327

```
function _swapUsdmToUsdt(uint256 amountIn) internal returns (uint256
    amountOut) {
     usdm.forceApprove(address(swapRouter), amountIn);

     // Scale minOut from USDm decimals  asset decimals, then apply
        slippage
     uint256 minOut = (amountIn * (10000 - MAX_SLIPPAGE_BPS)) / 10000 /
        DECIMAL_SCALE;
```

File: AvonStrategy.sol#L227-L233

```
uint256 neededUsdm = needed * DECIMAL_SCALE;
sharesToRedeem = megaVault.convertToShares(neededUsdm);
if (sharesToRedeem > maxShares) sharesToRedeem = maxShares;
if (sharesToRedeem > 0) {
     usdmRedeemed = megaVault.redeem(sharesToRedeem, address(this),
        address(this));
     if (usdmRedeemed > 0) {
          _swapUsdmToUsdt(usdmRedeemed);
     }
}
```

File: AvonStrategy.sol#L73-L80

```
uint256 private constant DECIMAL_SCALE = 1e12;
uint256 public constant MAX_SLIPPAGE_BPS = 50;
```

Reference (asymmetric mirror path): File: AvonStrategy.sol#L291-L297

```
function _swapUsdtToUsdm(uint256 amountIn) internal returns (uint256
    amountOut) {
     asset.forceApprove(address(swapRouter), amountIn);
     uint256 minOut = (amountIn * DECIMAL_SCALE * (10000 -
        MAX_SLIPPAGE_BPS)) / 10000;
```

Impact

Dust-sized USDm swaps can execute with no minimum received amount protection, enabling up to 100% loss on those dust events.

The per-event loss is strictly bounded to sub-cent magnitude due to decimal granularity, so the economic impact is low. However, the configured slippage guard is effectively disabled for this dust band.

Proof of Concept

```
// Constants in AvonStrategy:
// DECIMAL_SCALE = 1e12
// MAX_SLIPPAGE_BPS = 50

function poc_minOut_zero_for_dust() external pure returns (uint256 minOut
    ) {
    uint256 amountIn = 1_005_025_125_628; // just below threshold (USDm
        wei)
    minOut = (amountIn * (10000 - 50)) / 10000 / 1e12;
    // minOut == 0
}

function poc_minOut_one_at_threshold() external pure returns (uint256
    minOut) {
    uint256 amountIn = 1_005_025_125_629; // first value producing non-
        zero minOut
    minOut = (amountIn * (10000 - 50)) / 10000 / 1e12;
    // minOut == 1
}
```

Operational scenario: 1. Strategy withdraw flow redeems a dust USDm amount from `MegaVault`. 2. `_swapUsdmToUsdt()` sets `amountOutMinimum = 0`. 3. Swap can settle at `0` units out without violating `minOut`.

Recommendation

Do not force `minOut = 1` unconditionally for any non-zero input, because fair output can legitimately floor to `0` for tiny amounts and would then revert.

Safer options:
1. Skip swap when computed `minOut == 0` and accumulate dust until it crosses a configurable threshold.
2. Add a minimum input guard for `_swapUsdmToUsdt()` (or in `withdraw()`) so only swappable amounts are routed.
3. Optionally emit a `DustSwapDeferred(amountIn)` event for observability when dust is skipped.

Example pattern:

```
uint256 minOut = (amountIn * (10000 - MAX_SLIPPAGE_BPS)) / 10000 /
    DECIMAL_SCALE;
if (minOut == 0) {
    return 0; // defer dust conversion until enough accumulates
}
```

Team Response

Fixed.

[L-03] Emergency NFT Recovery Allows Extraction of Active Portfolio Positions

Severity

Low Risk

Description

The `emergencyRecoverNft()` is documented as a rescue function for non-portfolio NFTs accidentally sent to the strategy. However, it accepts any NFT contract and ID, and when the target is a tracked portfolio position, it explicitly calls `_removeNft()` before transferring, with no guard distinguishing expired or invalid positions from active ones backed by real depositor capital.

A strategy owner can call this function against any live lender NFT, removing it from all accounting structures and transferring it to an arbitrary address in a single transaction.

Attack flow:

1. Strategy holds NFT # 42 representing 50,000 USDT deployed into a `PropertyVault`. `config.currentDebt = 50,000`.
2. Strategy owner calls `emergencyRecoverNft(portfolioNFT, 42, attacker)`.
3. `_removeNft(42)` clears `heldNftIds` and `isHeldNft[42]`.
4. NFT is transferred to `attacker`, who now controls the `PropertyVault` lender position.
5. Next `processReport()` : `totalAssets()` returns 0 for the strategy (no tracked NFTs). Loss of 50,000 USDT is recognised and socialised across all shareholders. If loss > `maxLossReportBps`, the health check blocks even this recognition (see M-01) and `totalDebt` remains permanently inflated.

The same code path is present in `emergencyTransferNft()`, which operates identically but takes only `nftId` instead of an arbitrary contract address.

Location of Affected Code

File: RealEstateStrategy.sol#L423

```
function emergencyRecoverNft(address nftContract, uint256 nftId, address
    to) external onlyOwner {
    if (nftContract == address(0)) revert ZeroAddress();
    if (to == address(0)) revert ZeroAddress();

    if (nftContract == address(portfolioNFT) && isHeldNft[nftId]) {
        _removeNft(nftId);   // removes from heldNftIds and isHeldNft
            mapping
    }

    IERC721(nftContract).safeTransferFrom(address(this), to, nftId);
    emit EmergencyNftTransferred(nftId, to);
}
```

File: RealEstateStrategy.sol#L406

```
function emergencyTransferNft(uint256 nftId, address to) external
    onlyOwner {
    if (to == address(0)) revert ZeroAddress();

    if (isHeldNft[nftId]) {
        _removeNft(nftId);
    }

    IERC721(address(portfolioNFT)).safeTransferFrom(address(this), to,
        nftId);
    emit EmergencyNftTransferred(nftId, to);
}
```

Impact

The strategy owner can extract any active portfolio lender position in a single call. Each extracted NFT directly reduces `totalAssets()` by its full principal value. On the next `processReport()`, the loss is socialised across all vault depositors. `totalDebt` becomes permanently phantom debt with no recoverable backing.

A vault with a single 50,000 USDT strategy position can be zeroed out entirely by one owner call. The `to` parameter is unconstrained — funds can be redirected to any wallet, including one under the strategy owner's control.

Proof of Concept

```
// forge test --match-contract M06_EmergencyRecoverNftActivePosition_PoC
   -vv

function test_PoC_emergencyRecoverNft_extracts_active_portfolio_position
   () public {
    // Pre: strategy holds NFT #1 — 50,000 USDT lender position
    assertTrue(strategy.isHeldNft(NFT_ID),      "pre: strategy tracks NFT
        #1");
    assertEq(strategy.heldNftCount(), 1,          "pre: one active
       position");
    assertEq(portfolio.ownerOf(NFT_ID), address(strategy), "pre: strategy
        owns NFT");
    assertEq(strategy.totalAssets(), 50_000e6,   "pre: 50,000 USDT
       reflected in NAV");

    // Exploit: owner extracts the active position, no guard fires
    vm.prank(OWNER);
    strategy.emergencyRecoverNft(address(portfolioNFT), NFT_ID, ATTACKER)
       ;

    // Post: accounting wiped, NFT transferred to attacker
    assertFalse(strategy.isHeldNft(NFT_ID),      "post: position removed
       from tracking");
    assertEq(strategy.heldNftCount(), 0,          "post: no tracked
       positions remain");
    assertEq(portfolioNFT.ownerOf(NFT_ID), ATTACKER, "post: NFT owned by
       attacker");
}

function test_PoC_totalAssets_drops_to_zero_after_extraction() public {
    assertEq(strategy.totalAssets(), 50_000e6, "pre: 50,000 USDT");

    vm.prank(OWNER);
    strategy.emergencyRecoverNft(address(portfolioNFT), NFT_ID, ATTACKER)
       ;

    assertEq(strategy.totalAssets(), 0,
        "post: entire 50,000 USDT lender position wiped from accounting")
           ;
}
```

Recommendation

Add a guard that prevents `emergencyRecoverNft()` (and `emergencyTransferNft()`) from operating on active tracked positions without an explicit acknowledgement path:

```
// Prevent extraction of live portfolio positions via the recovery path
if (nftContract == address(portfolioNFT) && isHeldNft[nftId]) {
    revert CannotRecoverActivePosition(nftId);
}
```

If intentional extraction of an active position is ever required (e.g. after `PropertyVault` insolvency), split the function into two distinct entry points: - `emergencyRecoverNft()` — non-portfolio NFTs only (reverts if `nftContract == portfolioNFT`) - `emergencyEvictActivePosition()` — portfolio NFTs only, protected by vault owner co-signature or a time-lock to give depositors the ability to exit before the extraction settles

Team Response

Fixed.

[I-01] Missing Explicit `approve(0)` Sanitization Before `forceApprove()`

Severity

Informational Risk

Description

`forceApprove()` is used without an explicit `approve(0)` sanitization step before. For clear allowance hygiene, force-reset to `0` first as a best practice.

Location of Affected Code

File: src/strategies/AvonStrategy.sol

```
usdm.forceApprove(address(megaVault), usdmReceived);
asset.forceApprove(address(swapRouter), amountIn);
usdm.forceApprove(address(swapRouter), amountIn);
```

Impact

Potential allowance-handling ambiguity and reduced clarity of approval sanitization flow.

Recommendation

Explicitly set allowance to `0`, keeping sanitization clear.

Team Response

Fixed.

[I-02] Misleading Custom Error Name in `resolveEmergencyShutdown()`

Severity

Informational Risk

Description

The `resolveEmergencyShutdown()` function reverts when `emergencyShutdown` is `false`, meaning emergency shutdown is **not** active. However, it uses the custom error `EmergencyShutdownActive()`, whose name implies the opposite condition.

This creates a semantic mismatch between the revert condition and the error name. While the function logic is correct, the revert selector is misleading and may cause confusion for developers, auditors, and off-chain tooling that interprets custom errors by name.

Location of Affected Code

File: TulpeaYieldVault.sol#L482-L486

```
function resolveEmergencyShutdown() external onlyOwner {
    if (!emergencyShutdown) revert EmergencyShutdownActive();
    emergencyShutdown = false;
    emit EmergencyShutdownResolved();
}
```

Impact

There is no direct security impact. However, the incorrect error name can:

- mislead developers during debugging and maintenance,
- produce confusing revert traces,
- cause static analysis tools, monitoring systems, or off-chain parsers to display the wrong semantics.

Recommendation

Replace `EmergencyShutdownActive()` with an error name that accurately reflects the revert condition, such as:

```
error EmergencyShutdownNotActive();
```

and update the guard accordingly:

```
function resolveEmergencyShutdown() external onlyOwner {
    if (!emergencyShutdown) revert EmergencyShutdownNotActive();
    emergencyShutdown = false;
    emit EmergencyShutdownResolved();
}
```

Team Response

Fixed.

[I-03] `FundsDeployedToStrategy` Event Defined but Never Emitted

Severity

Informational Risk

Description

`FundsDeployedToStrategy` is declared in the vault events list, but is never emitted in the deployment execution path. `executeDeploy()` emits `DeploymentExecuted` only.

This creates an observability mismatch for monitors or analytics that subscribe to `FundsDeployedToStrategy`, expecting strategy deployment activity.

Location of Affected Code

File: TulpeaYieldVault.sol#L659

```
event FundsDeployedToStrategy(address indexed strategy, uint256 amount);

function executeDeploy(uint256 deploymentId) external onlyOwner
    nonReentrant whenNotPaused {
     // code
     IERC20(asset()).safeTransfer(pending.strategy, pending.amount);

     emit DeploymentExecuted(deploymentId, pending.strategy, pending.
        amount);
}
```

Impact

Off-chain indexers, bots, or dashboards listening for `FundsDeployedToStrategy` will miss deployment activity, which can cause incomplete monitoring and misleading operational telemetry.

Recommendation

Either remove the unused `FundsDeployedToStrategy` event to avoid confusion, or emit it alongside `DeploymentExecuted` inside `executeDeploy()` so event semantics stay consistent for integrators.

Team Response

Fixed.

[I-04] `requestDeploy()` and `executeDeploy()` Are Not Blocked During Emergency Shutdown

Severity

Informational Risk

Description

`requestDeploy()` and `executeDeploy()` are protected by `whenNotPaused`, but they are not protected by `emergencyShutdown()`.

The protocol already added `whenNotPaused` to both functions, which shows the intended operational gating for deployment actions. However, the emergency-specific guard was not added, so deployment requests can still be created and executed while `emergencyShutdown == true`.

Location of Affected Code

File: TulpeaYieldVault.sol#L659

```
function requestDeploy(address strategy, uint256 amount) external
    onlyOwner nonReentrant whenNotPaused returns (uint256 deploymentId) {
     // Missing: if (emergencyShutdown) revert EmergencyShutdownActive();
}

function executeDeploy(uint256 deploymentId) external onlyOwner
    nonReentrant whenNotPaused {
     // Missing: if (emergencyShutdown) revert EmergencyShutdownActive();
}
```

Impact

During emergency mode, capital can still be pushed into strategies through queued or newly requested deployments. This conflicts with the emergency objective of preserving capital and reducing exposure.

Recommendation

Add an emergency shutdown guard to both functions:

```
if (emergencyShutdown) revert EmergencyShutdownActive();
```

This aligns deployment behavior with emergency-mode expectations and prevents new strategy exposure while shutdown is active.

Team Response

Fixed.

[I-05] Inconsistency in How the Pausing Is Handled in Redeeming Processes

Severity

Informational Risk

Description

In `TulpeaYieldVault`, the redeeming process starts with requesting a redemption, then the redemption gets fulfilled by the owner. In between requesting and fulfilment, users could cancel the redemption with the `cancelWithdraw()` function. After the redemption gets fulfilled, the assets/shares are ready to be withdrawn/redeemed. During the paused state of the contract, new requesting and cancelling of already requested redeems are not allowed, but the fulfillment of the same is. This behavior creates an inconsistency since the already requested redeems can't be canceled, but at the same time they can still be fulfilled.

Location of Affected Code

File: TulpesYieldVault.sol#379-383

```
function requestRedeem(
    uint256 shares,
    address controller,
    address owner
) external nonReentrant whenNotPaused returns (uint256 requestId)
```

File: TulpesYieldVault.sol#408

```
function cancelWithdraw() external nonReentrant whenNotPaused
```

File: TulpesYieldVault.sol#432

```
function fulfillRedeem(address user) external onlyOwner nonReentrant
```

Impact

During the paused state (potentially invoked because of the exploit), users who have already requested redemption before the pausing can have their requests fulfilled, potentially at bad terms. At the same time, the cancellation of the same requests is not possible

Recommendation

Consider removing the `whenNotPaused` modifier from `cancelWithdraw()`, so the users/controllers can cancel it during the paused state or make fulfilment sensible to pausing as well.

Team Response

Fixed.

[I-06] Lack of Zero Address Check in `withdraw()`, `redeem()` and `_claimWithdrawal()` Functions

Severity

Informational Risk

Description

Functions `withdraw()`, `redeem()` and `_claimWithdrawal()` lack a sanity check if the receiver of the funds is a zero address, making it possible to withdraw assets to a zero address (owners/controllers should be trusted). This lack of check is in contradiction with the rest of the code, since other functions has it.

Location of Affected Code

File: TulpeaYieldVault.sol#283-295

```
function withdraw(uint256 assets, address receiver, address owner) public
    override nonReentrant returns (uint256) {
    if (assets == 0) revert ZeroAssets();
    uint256 claimable = claimableWithdrawals[owner];
    if (claimable == 0) revert NothingClaimable();
    if (assets > claimable) revert ExceedsClaimable(assets, claimable);
    // Skip mulDiv when claiming all remaining — prevents rounding dust
    uint256 shares =
        (assets == claimable)
            ? claimableWithdrawalShares[owner]
            : Math.mulDiv(claimableWithdrawalShares[owner], assets,
                claimable);
    _claimWithdrawal(owner, receiver, assets, shares);
    return shares;
}
```

File: TulpeaYieldVault.sol#298-309

```
function redeem(uint256 shares, address receiver, address owner) public
    override nonReentrant returns (uint256) {
    if (shares == 0) revert ZeroShares();
    uint256 claimable = claimableWithdrawals[owner];
    uint256 claimShares = claimableWithdrawalShares[owner];
    if (claimable == 0) revert NothingClaimable();
    if (shares > claimShares) revert ExceedsClaimable(shares, claimShares
        );
    // Skip mulDiv when claiming all remaining — prevents rounding dust
    uint256 assets = (shares == claimShares) ? claimable : Math.mulDiv(
        claimable, shares, claimShares);
    if (assets == 0) revert ZeroAssets();
    _claimWithdrawal(owner, receiver, assets, shares);
    return assets;
}
```

File: TulpeaYieldVault.sol#836-848

```
function _claimWithdrawal(address owner, address receiver, uint256 assets
    , uint256 shares) internal {
    if (msg.sender != owner && !isOperator(owner, msg.sender)) revert
        Unauthorized();

    claimableWithdrawals[owner] -= assets;
    claimableWithdrawalShares[owner] -= shares;
    totalClaimableWithdrawals -= assets;
    totalWithdrawn[owner] += assets;

    // Balance decreases naturally after safeTransfer
    IERC20(asset()).safeTransfer(receiver, assets);
    emit Withdraw(msg.sender, receiver, owner, assets, shares);
    emit RedeemClaimed(owner, assets);
}
```

Impact

Even though the owners/controllers should be trusted, it is still possible to withdraw/redeem funds to the zero address.

Recommendation

Consider adding a zero address check in both `redeem()` and `withdraw()` or only in the `_claimWithdrawal()` function.

Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!